OptiProductio growth roadmap: from one line to a mini smart factory

Project: OptiProductio, official title “Arduino-Based Real-Time WIP Monitoring and Bottleneck Detection” (IEM, IDEA Lab EL). Title updated 1 November 2026.

Version 2, 1 November 2026 (official title added; content unchanged from version 1 of 24 October 2026). Based on the industry platforms report v3. Phase II and III dates are not confirmed yet, so this plan assumes Phase II around January 2027 and Phase III around April 2027. Change the month targets once we have the real dates.

The short answer

The big platforms (Siemens, AVEVA, Velotic, AWS) all sell the same handful of jobs: watch the line live, find what is losing time, predict trouble, test changes on a model before trying them for real, plan the work, and keep a clean record of everything [1][3][20][37]. OptiProductio already does the first job for a 3-station line. We can grow it by adding the other jobs one at a time, each one as a small module that a first-year team can build and explain.

We suggest **eight add-on modules** after the 1 November build. Each one copies one product category from the industry menu, uses cheap parts or free software, and has one measurable target. Together they turn OptiProductio from “a dashboard for one line” into “a teaching-scale copy of a smart factory software stack”, which is a much stronger Phase III outcome. It also gives the paper more to stand on.

The rule for every module: **it must work on our own line with our own data, and its target must be something we can measure ourselves.** No module depends on a company partnership, private data or paid cloud services.

Where we stand today

Be honest about this on every slide. As of 24 October 2026:

Part	Status
Arduino sketch (3 IR counters, breakdown/no-material/reject buttons, LEDs, buzzer, bin sensor, LCD)	Written, only syntax-checked. Not yet tested on real hardware. In progress
Serial logger (Python)	Written and tested with a fake Arduino. In progress until tested on the real board
Metrics code: cycle time, WIP over time, OEE proxy, downtime by reason, Little's law check	Written and tested on fake data. In progress
Streamlit dashboard, including Bosch data mode	Written. In progress
3D animated model of the line	Done (demo page)
Industry platforms study	Done (report v3)

So the 1 November build is still the priority. Nothing in this roadmap should start before the line runs end to end.

How each module maps to the industry menu

The screenshot menu you shared lists the product categories big vendors sell. Here is where each OptiProductio module sits. This mapping is our own reading of those categories.

Industry category	What the big vendors sell	OptiProductio module
Production operations & optimization	OEE, downtime and bottleneck tracking (Insights Hub, ThingWorx RTPPM and DPM) [3][22][23]	M1 Loss hunter
MES software	Records what was made, by whom, and traces each part [1]	M2 Part tracking and shift report
Digital twin / simulation	Process models for what-if testing (CSense, TwinMaker) [20][37]	M3 Live twin and what-if
Data integration / historian	Broker, Unified Namespace, historian [1][64]	M4 Wireless line and historian
Asset health / predictive maintenance	Predict machine trouble from sensor history [3][47]	M5 Machine health watch
Sustainability / energy KPIs	Energy per unit, part of ISO 22400 since 2017	M6 Energy per part

Industry category	What the big vendors sell	OptiProductio module
	[51]	
Production planning & scheduling	APS: who makes what, when, on which machine [7][58]	M7 Mixed-product scheduler
Analytics on history	Industrial AI analytics [47]	M8 Bosch data study (the paper)

The eight modules

Each module below has what it adds, what we need, what it costs (approximate online prices in rupees, check before buying), and its SMART target, written as a Project Outcome (PO).

M1 Loss hunter: Pareto, bottleneck finder and SPC

What it adds. Today the dashboard shows how much time was lost. M1 shows *why* and *where*. Each stop gets a reason code mapped to the six big losses [49]. A Pareto chart ranks the reasons. The active-period method marks which station is the bottleneck at each moment, using the station busy/idle times we already log [55][71]. An individuals control chart on cycle time and a p-chart on rejects flag when the line has changed [60].

What we need. Python only (pandas, plotly). There is a ready Python package for the active-period method [71]. Cost: nil.

Why it impresses. This is the core of what ThingWorx DPM and RTPPM sell: automatic bottleneck finding and ranked losses [22][23].

PO-M1. By end of November 2026, the dashboard shows a downtime Pareto, a live bottleneck marker and two control charts. Test: in a 30-minute run where we deliberately slow station 2, the bottleneck marker picks station 2 for at least __ % of the run (target to be set after the first test run).

M2 Part tracking and shift report (MES-lite)

What it adds. Give each part a number when it enters station 1, and record its time at each station. That gives per-part flow time and lets us trace any reject back to when it happened. A “shift report” button exports one table: planned time, downtime, good and rejected parts, availability, performance, quality, throughput, average WIP, MTBF and MTTR, using ISO 22400 KPI names [50][51].

What we need. Python only. Cost: nil.

PO-M2. By mid-December 2026, the shift report generates as a PDF or CSV in one click, and every KPI on it has its formula and data source written next to it.

M3 Live twin and what-if simulator

What it adds. Two things. First, the 3D model of the line becomes "live": it reads the latest data and colours each station green, amber or red, the way TwinMaker scenes change colour from live tags [37]. Second, a SimPy model of the same 3 stations, using cycle times measured from our own line [59]. A slider sets the WIP cap from 1 to 10, and the chart shows throughput and flow time for each cap. The "knee" of that curve is the suggested cap, which follows the Factory Physics idea that throughput stops rising after a certain WIP [54].

What we need. SimPy (free), and a small change to the 3D page so it reads a data file. Cost: nil.

Why it impresses. This is what vendors call a digital twin. Be careful with the word: a model only counts as a twin when it stays linked to live data. A model run on assumed data is a simulation [20].

PO-M3. By end of December 2026, the SimPy model, fed with our measured cycle times, predicts the real line's throughput within __ % (target: 10%) over three test runs at different WIP caps.

M4 Wireless line and historian (MQTT)

What it adds. Replace the USB cable with an ESP32 board that sends each event over Wi-Fi to a local MQTT broker (Mosquitto) on the laptop. Topic names follow ISA-95 levels, such as `rvce/idealab/line1/station2/count`, which is the Unified Namespace idea in the Siemens IT/OT framework [1][11]. All events go into a SQLite or InfluxDB "historian" [64]. If Wi-Fi drops, the ESP32 keeps events in memory and sends them later (store and forward) [1].

What we need. One ESP32 dev board (about ₹400–500), Mosquitto and SQLite (free). The Arduino UNO can stay as the sensor board and pass data to the ESP32, or the ESP32 can take over.

PO-M4. By end of January 2027, the line runs for 2 hours over Wi-Fi and the historian has lost fewer than 1% of events, checked against the LCD counters.

M5 Machine health watch

What it adds. Put a small DC motor (a mock conveyor or "drill") at station 2 and a vibration sensor (MPU6050 accelerometer) on it. The dashboard learns what normal vibration looks like and flags when it drifts, using a simple rule first (EWMA chart or z-score), not deep

learning. Separately, we train a small model on the public AI4I 2020 predictive maintenance dataset (licence CC BY 4.0) to show how failure prediction works on bigger data [62].

What we need. MPU6050 module (about ₹150–250), small DC motor with driver (about ₹200–300). Python scikit-learn (free).

Model training parameters (the Phase I evaluators asked for these):

Item	Value
Dataset	AI4I 2020 (UCI), 10,000 rows [62]
Inputs	Air temperature, process temperature, rotational speed, torque, tool wear
Label	Machine failure (yes/no)
Model	Logistic regression first, then a random forest
Split	80/20, stratified by label
Metrics	Precision, recall, F1 on the failure class
Target	F1 ≥ __ (set after a baseline run; do not quote a number before we have one)

PO-M5. By end of February 2027, the live vibration alert catches at least __ of 10 deliberate “fault” tests (for example, a small weight taped to the motor) with no more than __ false alarms per hour.

M6 Energy per part

What it adds. A current and voltage sensor (INA219) on the motor supply measures energy used. The dashboard shows energy per good part, and energy wasted while the line is stopped or blocked. ISO 22400 added energy KPIs in a 2017 amendment, so this follows a real standard [51].

What we need. INA219 module (about ₹150–250). Cost is small.

PO-M6. By end of February 2027, the dashboard shows energy per good part, and we report the difference between a run with the WIP cap and a run without it, with both numbers measured, not assumed.

M7 Mixed-product scheduler

What it adds. Make the line handle two or three product types (for example, coloured tokens with different times at each station). Then use Google OR-Tools to plan the order of a batch of jobs so the whole

batch finishes as early as possible, and compare it with simple rules like first-come-first-served (FIFO) and shortest-job-first (SPT). Show all three as Gantt charts [58].

What we need. OR-Tools (free); coloured tokens; the IR sensors already count them, and an operator button or a colour sensor (TCS3200, about ₹300–400) can tell the type.

PO-M7. By end of March 2027, for 3 test batches of 8 jobs, we report makespan for OR-Tools, FIFO and SPT, both planned and actually measured on the line.

M8 Bosch data study: the conference paper

What it adds. The research angle from report v1: rebuild WIP, flow time and throughput from the public Bosch production line data, test whether failure rate goes up with WIP, and backtest the same WIP-cap alert that runs on our line [52][65]. This is what turns the project into a publication.

PO-M8. Submit a 4-page Work-in-Progress paper to IEEE CONECCT 2027 (Bengaluru) by its deadline, expected around February 2027 (to be confirmed when the call for papers comes out).

Our own angle: the A/B test

The single strongest thing we can add is not a feature. It is a fair test. Run the line in the same way twice, once **without** the WIP cap and once **with** it, for the same time and the same operators, three times each. Report throughput, average flow time, WIP and rejects for both. None of the vendor pages we read publish a baseline for their improvement claims [20][23]. A small but honest before/after table from our own line is better evidence than those claims, and it is exactly what the “results and discussion” rubric line asks for.

PO-AB. By the Phase II review, we have a table of 3 runs without the cap and 3 runs with it, with every number measured. Blanks stay blank until we measure them.

Suggested order and timing

When (assumed)	Modules	Why this order
By 1 Nov 2026	Finish the base build	Everything else depends on it
Nov 2026	M1 Loss hunter, A/B test runs	Software only, uses data we already log
Dec 2026	M2 Shift report, M3 Live twin and what-if	Software only; M3 needs M1’s measured times

When (assumed)	Modules	Why this order
Jan 2027	M8 paper analysis and writing, M4 Wireless line	Paper deadline around Feb; M4 is the first new hardware
Feb 2027	M5 Machine health, M6 Energy	Cheap sensors, add after M4 is stable
Mar 2027	M7 Scheduler	Needs multiple product types on the line
Apr 2027 (Phase III)	Polish, open-source release, report, poster	Demo the full stack

If time runs short, cut from the bottom. M1, M3 and the A/B test alone already make a strong Phase II.

Front end: use Velotic as the reference

When we redesign the dashboard for these modules, use **velotic.com** as the design reference, as you asked. Copy its structure, not its brand: a one-line status headline, a row of big-number tiles, then charts. Use tabs named with verbs like Velotic's CSense sequence: **Monitor, Analyze, Predict, Simulate, Plan** [20]. Each module above fits one tab. Do the browser checklist in report v3 to note Velotic's colours and fonts, then set them in Streamlit's theme file.

What we should still not add

- Paid cloud platforms (Insights Hub, TwinMaker, ThingWorx). Azure IoT Central is closed to new apps anyway [44].
- Agentic AI or chatbots. Verdantix lists these as new market categories, but they add little for a 3-station line and are hard to explain to judges [47].
- Real PLCs, industrial networks or enterprise security products. These solve problems we do not have.
- A patent. Software on its own usually cannot be patented in India. The realistic outcomes are the conference paper and an open-source prototype.

Total extra cost

Item	Approx. price (check before buying)
ESP32 dev board	₹400–500
MPU6050 accelerometer	₹150–250
Small DC motor + driver	₹200–300
INA219 current sensor	₹150–250
TCS3200 colour sensor (optional)	₹300–400

Item	Approx. price (check before buying)
Total	**about ₹1,200–1,700**

All software (Python, Streamlit, SimPy, OR-Tools, scikit-learn, Mosquitto, SQLite) is free.

References

[1] Siemens AG. (2026). *IT/OT framework* (Application example, Entry ID 109998257, V1.0) [PDF]. Siemens Industry Online Support. https://support.industry.siemens.com/cs/ww/en/view/109998257

[3] Siemens. (n.d.). *Insights Hub*. https://www.siemens.com/en-us/products/insights-hub/

[7] Siemens Digital Industries Software. (n.d.). *Opcenter APS* [E-book]. https://assets.new.siemens.com/siemens/assets/api/uuid:87f07fef-52c5-4349-a7f4-95f5733a0646/siemens-sw-opcenter-aps-e-book.pdf

[11] Siemens. (n.d.). *ISA-95 framework and layers*. https://www.siemens.com/en-us/technology/isa-95-framework-layers/

[20] Velotic. (n.d.). *Proficy CSense*. https://www.velotic.com/products/proficy/csense

[22] Velotic. (n.d.). *ThingWorx Digital Performance Management*. https://www.velotic.com/products/thingworx/digital-performance-management

[23] Velotic. (n.d.). *ThingWorx Real-time Production Performance Monitoring*. https://www.velotic.com/products/thingworx/real-time-production-performance-monitoring

[37] AWS Samples. (n.d.). *aws-iot-twinmaker-samples: cookiefactoryv2* [Source code]. GitHub. https://github.com/aws-samples/aws-iot-twinmaker-samples/tree/main/src/workspaces/cookiefactoryv2

[44] Microsoft. (2025–2026). *Azure IoT Central documentation* (“Azure IoT Central is evolving”, updated 23 September 2026). https://aka.ms/AzureIoTCentral

[47] Kirkman, H., Devgun, J., & Graessle, J., with Tohani, M. (2025). *Green quadrant: Industrial AI analytics software (2025)* [ABB reprint]. Verdantix.

[49] Nakajima, S. (1988). *Introduction to TPM: Total productive maintenance*. Productivity Press. (unverified, check before citing)

[50] Fabrico. (n.d.). *OEE and ISO 22400*. https://www.fabrico.io/blog/oee-iso-22400/ (vendor blog)

[51] International Organization for Standardization. (2014). *Automation systems and integration: Key performance indicators (KPIs) for manufacturing operations management. Part 2: Definitions and descriptions* (ISO 22400-2:2014). https://www.iso.org/standard/54497.html

[52] Spearman, M. L., Woodruff, D. L., & Hopp, W. J. (1990). CONWIP: A pull alternative to kanban. *International Journal of Production Research, 28*(5), 879–894. https://doi.org/10.1080/00207549008942761

[54] Hopp, W. J., & Spearman, M. L. (2008). *Factory physics* (3rd ed.). Waveland Press. (unverified, check before citing)

[55] Roser, C. (n.d.). *The active period method*. AllAboutLean. https://www.allaboutlean.com/active-period-method

[58] Google. (n.d.). *The job shop problem*. OR-Tools. https://developers.google.com/optimization/scheduling/job_shop

[59] SimPy Team. (n.d.). *SimPy documentation*. https://simpy.readthedocs.io/en/latest/

[60] Montgomery, D. C. (n.d.). *Introduction to statistical quality control*. Wiley. (edition and year unverified, check before citing)

[62] Matzka, S. (2020). *AI4I 2020 predictive maintenance dataset* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C5HS5C (author unverified, check before citing)

[64] Arduino. (n.d.). *Data logging with MQTT, Node-RED, InfluxDB and Grafana*. https://docs.arduino.cc/tutorials/portenta-x8/datalogging-iot

[65] Kim, S.-H., & Whitt, W. (2013). Statistical analysis with Little’s law. *Operations Research, 61*(4), 1030–1045. https://doi.org/10.1287/opre.2013.1193

Reference numbers match the industry platforms report v3, so the two documents can share one reference list.